¿Qué es Flyway y por qué no está en el diagrama de arquitectura?

Asunto: cómo se crea el esquema de `auth_catalogs_db` (ms-auth-catalogs) · **Fecha:** 25 de septiembre de 2026 · Avance 1
Fuente: estado real del repositorio, rama `main`, commit `c3686ed`

1 Qué es Flyway

Flyway **no es un componente de la arquitectura**. Es una librería que va dentro del jar de `ms-auth-catalogs`, al mismo nivel que Spring Data JPA o el driver de MySQL — que tampoco aparecen en el diagrama.

- Hace una sola cosa: **aplicar scripts SQL versionados al arrancar el servicio**.
- Los scripts viven en `src/main/resources/db/migration/`: `V1__esquema_inicial.sql` (crea las 11 tablas) y `V2__hu024_permiso_recibido_exclusivo.sql` (corrige un permiso ya sembrado).
- Lleva el control en su propia tabla `flyway_schema_history`: mira qué versión tiene la base, **ejecuta solo las migraciones que faltan y nunca repite una ya aplicada**.
- En una frase: es *git para el esquema de la base*. La base vacía de local y la de RDS terminan con el mismo esquema, sin pegar un `.sql` a mano en Workbench.

2 Por qué no venía en la arquitectura

Revisado el 25/09 en los tres entregables del Avance 1:

DOCUMENTO	MENCIONES A FLYWAY, MIGRACIONES, DDL O MODELO DE DATOS
`Diagrama arquitectura - Neptuno.drawio`	0
`Neptuno_Proyecto.docx`	0
`Neptuno_Requerimientos_Backlog_Sprints.xlsx`	0

Es cierto: no venía. Pero tampoco venían JPA/Hibernate, la librería de JWT, **nginx** ni el driver JDBC, y nadie los trata como desviación: el diagrama dibuja **componentes** (servicios, broker, bases, MinIO, EC2 y RDS), no las librerías internas de cada servicio.

Flyway no altera nada de lo dibujado:

- No es un servicio nuevo ni un contenedor nuevo.
- No tiene costo y no agrega comunicación entre microservicios.
- `auth_catalogs_db` sigue siendo **una sola base para ms-auth-catalogs**, dentro del *«Amazon RDS (cluster único) · MySQL · una BD por microservicio»*.

Cambia el *cómo* se crean las tablas, no el *qué*.

3 El diagrama sí obliga a resolver el «cómo»

El diagrama es explícito: **una BD por microservicio**, con `auth_catalogs_db` nombrada. Alguien tiene que crear esas tablas. Las alternativas:

OPCIÓN	PROBLEMA
Pegar los `create table` a mano	No reproducible ni auditable: se pierde cuando alguien recrea la base.
`ddl-auto: update` de Hibernate	Hibernate «adivina» el esquema; no versiona ni documenta los cambios. No se recomienda en producción.
Migraciones versionadas (Flyway)	**El esquema viaja con el código, se aplica igual en local y en RDS, y cada cambio queda registrado.**

Por eso el servicio arranca con `ddl-auto: validate`: Hibernate no crea ni un índice, solo valida que las entidades coincidan con el esquema que aplicó Flyway.

Esquema actual: 11 tablas en `auth_catalogs_db`, más `flyway_schema_history`.

Identidad / RBAC	`usuario`, `rol`, `permiso`, `usuario_rol`, `rol_permiso`, `area`, `token_revocado`
Catálogos maestros	`tipo_documental`, `tipo_documental_origen`, `entidad`
Auditoría	`evento_auditoria`

4 Respuesta de una frase

> «El esquema no se crea a mano: viaja versionado con el código del servicio. En ms-auth-catalogs eso lo hace Flyway; cuando montemos RDS corre las mismas migraciones y las tablas se crean solas. Por eso no hay un `.sql` suelto en el repositorio.»

5 Pendiente de equipo

Cada servicio necesita su equivalente: **ms-documental** (Node/Express), **ms-bpm** (NestJS) y **ms-audit-infra** (Go). Prisma, Drizzle, TypeORM o `golang-migrate` cumplen el mismo papel. Si no se estandariza, cada servicio inventará su forma de crear el esquema, y ahí el diagrama sí se queda corto.

Evidencia citada: `servicio-usuarios/target/verificacion.log` (25/09, 20:05) — Flyway conectado a MySQL 8.4, migración validada y esquema en la versión 1; hoy el repositorio tiene 2 migraciones. Suite del servicio: `mvn test` → 35/35 pruebas en verde (25/09, 21:33).

> **Cómo se lee esto en el video:** el guion ya dice «migraciones Flyway para la base de datos». Esta nota solo explica el detalle de implementación; no contradice el diagrama de arquitectura ni agrega infraestructura al Avance 1.